EL PANTEÓN

Apéndice Arquitectónico y Manual Operativo v3.0

> **Visión Core:** "Sin plantillas estáticas, sin Hype, sin herramientas de caja negra."
>
> El Panteón es un motor de renderizado y orquestación audiovisual desarrollado en **Go**. Su propósito es producir vídeos técnicos (10-12 minutos) sobre Arquitectura de Software e Inteligencia Artificial, compuestos por clips atómicos procedurales (15-20s), generando Motion Graphics de alto rendimiento en Chromium sin intervención humana.

1. La Jerarquía Audiovisual (Fractal)

El sistema no escupe clips al azar, ni procesa un vídeo de 12 minutos en un solo pase. Utiliza una arquitectura en 3 niveles para proteger la memoria RAM del VPS (OCI/Hetzner) y permitir la paralelización:

- **Nivel 1: El Clip Atómico (15-20 segundos).** La unidad física de compilación y renderizado. Combina una locución concreta, un árbol DOM generado al vuelo, un estado de animación CSS y efectos de sonido.
- **Nivel 2: La Escena (1 a 3 minutos).** Secuencia de 4 a 9 clips unidos lógicamente. Mantiene la continuidad atmosférica y la música de fondo constante. *(Ej: "La Autopsia del Kernel")*.
- **Nivel 3: El Vídeo Maestro (10-12 minutos).** Ensamblaje final de ~40 clips atómicos mediante `FFmpeg Concat Demuxer` en modo stream copy (cero recodificación).

> **Ventaja de la Caché (Idempotencia):** Si el clip #14 necesita una corrección de guion, el orquestador SHA-256 en Go solo re-renderiza esos 15 segundos y concatena de nuevo. Costo de iteración: casi cero.

2. El Monorepo (Topología del Sistema)

La arquitectura adopta un enfoque de Monorepo para consolidar recursos, pero manteniendo los entornos de ejecución aislados mediante contenedores.

```
el-panteon/
├── services/
│   └── pavarotti/            # Microservicio de síntesis TTS (Chatterbox/Python)
├── engine/                   # Orquestador y renderizador maestro (Go)
│   ├── internal/
│   │   ├── galileo/          # Rastreador Anti-Hype (Cazador de problemas)
│   │   ├── atlas/            # Extractor de Repositorios y Docs
│   │   ├── socrates/         # Validador de Rigor Técnico
│   │   ├── cervantes/        # Guionista omnisciente (Emisor del DirectorCut)
│   │   ├── davinci/          # Motor procedural de HTML/SVG sin plantillas
│   │   ├── pixar/            # Render Determinista (Virtual Time) a 60 FPS
│   │   └── mozart/           # Mezclador FFmpeg (SFX + Auto-ducking)
├── shared/
│   ├── contracts/            # Esquemas JSON (EngineeringCase, DirectorCut)
│   └── assets/               # Logos (SVGs), Música, Tipografías
└── docker-compose.yml
```

3. Los Flujos de Datos Clave

A. El Pipeline de Inteligencia (De la Web al Guion)

Componente	Entrada	Lógica Interna	Salida
Galileo	Fuentes primarias (RFCs, Post-Mortems)	Aplica filtro Anti-Hype. Busca la "Tríada": Síntoma, Causa, Solución.	`engineering_case.json`
Atlas	`engineering_case.json`	Descarga código real de GitHub, diagramas y benchmarks de referencia.	`dossier_tecnico.json`
Cervantes	`dossier_tecnico.json`	Desglosa el documental en escenas y define la intención gráfica.	`director_cut.json` (x40)

B. El Pipeline de Renderizado (Del JSON al MP4)

```
                [ Cervantes ] -- JSON --> [ Orquestador GO ]
                                          ↓
      1. Llama a Pavarotti (Chatterbox HTTP) --> .wav + Duración
2. Llama a Da Vinci --> Construye DOM Procedural adaptado al Canvas.
3. Invoca a Pixar (Chromium Headless) --> Avanza DOM 16.6ms / frame.
        4. Graba Pipe a FFmpeg --> MP4 crudo (60FPS Exactos).
           5. Llama a Mozart --> Mezcla audio con ducking.
                                          ↓
              [ FFmpeg Concat ] -- Ensambla 40 clips --> Master.mp4
```

4. Adiós a las Plantillas: Da Vinci & Pixar

En lugar de utilizar plantillas HTML fijas donde el texto se desborda, **Da Vinci opera como un solucionador de restricciones espaciales (Constraint-Based UI)**.

- **Composición al vuelo:** Si Cervantes pide "Comparativa de 3 bases de datos", Da Vinci no carga `plantilla_3.html`. Instancia 3 primitivas Atómicas (Cards), calcula un `flex-box` equitativo en un canvas estricto de 1920x1080, y traza las líneas de convergencia SVG usando matemáticas absolutas.
- **Reloj Virtual Determinista (Pixar):** Pixar no captura la pantalla en tiempo real (lo que causaría *lag* o pérdida de fotogramas en el VPS). Usa `go-rod` para inyectar `window.advanceAnimation(16.6ms)`, obligando a las animaciones de interpolación CSS/JS a saltar al frame exacto, extrayendo un PNG perfecto antes de avanzar al siguiente milisegundo.

5. Siguiente Hito Inmediato

El desarrollo debe comenzar por la validación del bloque atómico. La prioridad número uno es:

Prueba de Concepto Vertical (Clip de 15 Segundos):
1. Inyectar un JSON inventado (Ej: 4 tarjetas rojas de error).
2. Hacer un POST HTTP en Go hacia el contenedor Pavarotti (Easypanel) para obtener el `.wav`.
3. Generar el HTML con Da Vinci.
4. Renderizar un `clip_01.mp4` de 60FPS inyectado en FFmpeg vía pipes.